

Build Your Own Internal Tools

A practical guide to replacing SaaS bloat with tools that actually fit

By SmplCo

smpl.as

Why Build Your Own?

The SaaS landscape has a problem: you're paying for 10+ tools and using 10% of each one. Here's why building your own makes sense.

- **Nothing connects.**
You spend more time switching tabs, copy-pasting data, and working around tool limitations than actually working. Your CRM doesn't talk to your planner. Your email tool doesn't know about your pipeline.
- **Every change costs money.**
Need a custom field? Upgrade. Need an integration? Plugin. Need a workflow change? Workaround. The cost of adapting generic platforms adds up fast.
- **The economics have shifted.**
Building used to mean hiring a dev team for months. Now, with AI-assisted coding, modern frameworks, and generous free tiers, a competent team can build a tailored internal tool in days, not months.
- **You own it.**
No vendor lock-in. No surprise price hikes. No features removed. No waiting for a roadmap that never arrives. Your tool, your rules.

The cost of generic platforms keeps going up.

The cost of building keeps going down.

What's Actually Easy to Build

You'd be surprised what a small team (or even one person with an AI coding assistant) can ship in a week or two. Here's a realistic breakdown:

TOOL	WHAT IT INCLUDES	TIME ESTIMATE
Simple CRM	Contact tracking, pipeline stages, activity log	1-2 weeks
Task Planner / Team Scheduler	Drag-and-drop tasks, assignments, deadlines	1 week
Newsletter / Email Tool	Template editor, send/schedule, analytics	1 week
Internal Dashboard	KPIs, charts, real-time data from your DB	2-3 days
Landing Page Generator	Template system, form builder, deploy	1 week
Admin Panel	CRUD interface for your database tables	2-3 days
Client Portal	Login, project status, file sharing, messaging	1-2 weeks
Reporting Dashboard	API integrations, data viz, scheduled reports	1-2 weeks

These estimates assume one developer using AI-assisted coding (Claude Code, Cursor, or similar). Traditional development would take 2-3x longer.

The Stack

You don't need a complex infrastructure. This stack is battle-tested, mostly free, and scales well:

■ Database: Supabase (Postgres)

Free tier is generous. Full Postgres under the hood. Real-time subscriptions, row-level security, built-in auth. Scales from prototype to production without switching providers.

■ Frontend: Next.js + React

Industry standard. Massive ecosystem. Server-side rendering, API routes, file-based routing. Your AI coding tool knows it inside and out.

■ AI Coding Assistant

This is the game changer. Claude Code, Cursor, or similar tools let you describe what you want and get working code. They handle the boilerplate. You focus on the logic.

■ Design: Figma

For your style guide, components, and design system. Even a simple Figma file with your colors, fonts, and button styles saves hours of inconsistency.

■ Hosting: Vercel

Auto-deploys from Git. Generous free tier. Global CDN. SSL. Preview deployments for every branch. Zero-config for Next.js.

■ Auth: NextAuth.js or Supabase Auth

Don't build auth from scratch. Both options give you OAuth, magic links, and session management out of the box.

Using Figma for Your Style Guide

A simple design system prevents your tools from looking like they were built by five different people on five different days. Start small.

- **Start with the basics**
Define your colors, typography scale (3-4 sizes), spacing units (4px, 8px, 16px, 24px, 32px), and border radius. This takes 30 minutes and saves you hours.
- **Build reusable components**
Buttons (primary, secondary, ghost), cards, inputs, modals, navigation. 10-15 components covers 80% of internal tools.
- **Use Figma variables**
Set up color tokens and spacing tokens as Figma variables. This makes light/dark mode trivial and keeps everything consistent.
- **Reference designs in your AI tool**
Your AI coding tool can work from Figma designs. Screenshot a component, describe the layout, and let the AI generate matching code.
- **Keep it simple**
Internal tools don't need to win design awards. They need to be clear, consistent, and fast to use. A minimal design system achieves this.

PRO TIP

Don't design everything upfront. Build your design system as you build your first tool. Extract patterns after you see them, not before.

Using Claude Code (or Similar) to Build

AI coding assistants have fundamentally changed what a small team can build. Here's how to get the most out of them:

- **Be specific in your prompts**

"Build a contact list" is vague. "Build a contact table with columns for name, email, company, status (lead/active/churned), and last contacted date, with sorting and search" gets you something usable.

- **Start with the database schema**

Get your data model right first. Everything else flows from it. Describe your tables, relationships, and constraints clearly.

- **Build one feature at a time**

Don't try to build everything in one go. Create the contact list. Test it. Then add the pipeline view. Then add email integration. Each piece should work before you move on.

- **Let AI handle the boring parts**

CRUD operations, form validation, data fetching, pagination, search, sorting — these are solved problems. Let the AI write them. You focus on the business logic and workflows that are unique to your company.

- **Iterate, don't perfect**

The first version won't be perfect. Ship it, use it, and improve it. A tool that works today beats a perfect tool that ships next month.

- **Provide context**

Give the AI your existing code, your database schema, your design tokens. The more context it has, the more consistent the output.

What to Watch Out For

Building your own tools is powerful, but it comes with responsibilities. Keep these in mind:

Scope creep

The number one killer. Build what you need today, not what you might need someday. You can always add features later. You can't get back the time spent building things nobody uses.

Maintenance burden

You're the support team now. Every feature you add is a feature you maintain. Keep your codebase simple and well-documented. Future you will thank present you.

Don't rebuild everything at once

Start with one tool. The most painful one, or the one where you're wasting the most money. Get that working well before you move to the next one.

Security

Use proper authentication. Don't store secrets in code. Use environment variables. Enable row-level security in Supabase. Don't skip this.

Know when NOT to build

Some things should stay on established platforms: payment processing (Stripe), complex accounting (Xero/QuickBooks), legal compliance tools, anything where a bug could cost you money or legal trouble.

Getting Started: A 5-Day Plan

Stop planning. Start building. Here's your first week:

DAY 1

Audit Your Tools

List every SaaS tool your team uses. For each one, write down: what you actually use it for, what percentage of features you use, what it costs, and what's annoying about it. Be honest.

DAY 2

Pick ONE Tool to Build

Choose the tool that's most painful or most wasteful. Not the most complex one — the one where the gap between what you need and what you have is biggest. This is your first project.

DAY 3

Set Up the Stack

Create your Supabase project, scaffold a Next.js app, connect it to Vercel. Design your data model — tables, columns, relationships. This is the foundation. Get it right.

DAY 4

Build the Core Feature

Use your AI coding assistant to build the main feature. Not all the features — just the core one. The thing that makes this tool worth using. Focus and ship.

DAY 5

Test, Ship, Iterate

Get your team using it. Collect feedback. Fix the obvious issues. Deploy it. Schedule next week's improvements. You now have a custom tool that fits your workflow perfectly.

SmplCo Can Help

We've built 125+ digital products for startups, scaleups, and global brands. We know what works, what doesn't, and how to move fast without cutting corners.

What we offer:

■ 5-Day Prototyping Service

Go from idea to working prototype in one week. We scope, design, build, and ship.

■ AI-Assisted Development

We use Claude Code and modern AI tools to build faster and smarter. You get production-quality code at startup speed.

■ Design Systems & Component Libraries

We create Figma design systems and matching React component libraries. Build consistently, ship faster.

■ Custom Internal Tools

Don't want to build it yourself? We'll build it for you. CRMs, dashboards, admin panels, client portals — whatever you need.

Ready to build?

Get in touch: smpl.as/contact

Visit us: smpl.as